EX09 - Presion dinamica con funciones - def, parametros y return

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex09_dynamic_pressure_function.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Resolver EX01 de forma reutilizable encapsulando validacion y calculo dentro de una funcion.

Variables / elementos del problema

Nombre	Significado
rho	densidad
speed	rapidez
q	presion dinamica

Ecuacion o logica en papel

```
q = 0.5 * rho * speed**2
```

Comprobacion manual

La fisica no cambia: rho=1.225 y speed=100 deben seguir dando 6125 Pa.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- variables
- if/elif/else
- presion dinamica

Nuevo en este ejercicio:

- def
- parametros
- argumentos
- return
- None
- llamada de funcion
- is not None
- reutilizacion

3. Conceptos Python nuevos

1. Definir funcion

Sintaxis: `def calculate_dynamic_pressure(rho, speed):`

def crea una funcion. rho y speed son parametros locales que recibiran valores cuando se llame.

2. return

Sintaxis: `return q`

return envia un valor al codigo que realizo la llamada. No es lo mismo que print().

3. None

Sintaxis: `return None`

None representa ausencia de un resultado util. Aqui se usa para entradas invalidas.

4. Llamada

Sintaxis: `calculate_dynamic_pressure(rho, speed)`

Los parentesis ejecutan la funcion y entregan argumentos a los parametros.

4. Codigo paso a paso

Paso 1. Definir la funcion

```
def calculate_dynamic_pressure(rho, speed):
```

Al ejecutar el archivo Python crea la funcion, pero todavia no calcula q.

Paso 2. Validar dentro

```
if rho < 0:
    return None
elif speed < 0:
    return None
```

La funcion se protege a si misma. Un return termina esa llamada inmediatamente.

Paso 3. Calcular y devolver

```
q = 0.5 * rho * speed**2
return q
```

q es local. return permite que su valor salga hacia el caller.

Paso 4. Llamar y guardar retorno

```
dynamic_pressure = calculate_dynamic_pressure(rho, speed)
```

El valor de return q pasa a dynamic_pressure.

Paso 5. Comprobar None

```
if dynamic_pressure is not None:
    print(...)
```

Solo presentamos la presion si la funcion realmente devolvio un resultado valido.

Paso 6. Reutilizar

```
q1 = calculate_dynamic_pressure(1.225, 100)
q2 = calculate_dynamic_pressure(1.225, 200)
```

La misma logica puede ejecutarse con nuevas entradas sin copiar el codigo interno.

5. Flujo de ejecucion

```
define function
 -> set inputs
 -> call function
 -> local validation/calculation
 -> return
 -> caller uses result
```

6. Resultado / comprobacion esperada

La llamada nominal devuelve 6125.0 Pa. Las llamadas extra producen 6125, 24500 y 50000 Pa.

7. Errores tipicos al empezar

- Confundir definir una funcion con ejecutarla.
- Olvidar return.
- Esperar usar fuera una variable local.
- No comprobar None.

8. Si solo recuerdas cinco cosas

- def crea la funcion.
- Los parametros estan en la definicion; los argumentos llegan en la llamada.
- return devuelve datos.
- Las variables locales viven dentro de la funcion.
- Una funcion evita duplicar logica.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.